

Lab 07: Open Ended

Student Information

- **Name:** Abdul Hayy Khan
- **Roll No:** 24F-AI-051

Project: 1. Employee Attrition Analyzer

Analyze a dataset of employee records to predict whether an employee is likely to leave the company.

Dataset: [IBM HR Analytics Employee Attrition & Performance](#)

Clean and preprocess categorical and numerical data.

```
# import libraries

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler

from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier

from sklearn.metrics import (
    accuracy_score,
    precision_score,
    recall_score,
    f1_score,
    confusion_matrix,
    classification_report,
    ConfusionMatrixDisplay
)

# load dataset

df = pd.read_csv("datasets/WA_Fn-UseC_-HR-Employee-Attrition.csv")

print("First 5 rows:")
print(df.head())

print("\nShape:")
print(df.shape)
```

```
print("\nDataset Information:")
print(df.info())
```

```
print("\nMissing Values:")
print(df.isnull().sum())
```

First 5 rows:

	Age	Attrition	BusinessTravel	DailyRate	Department
0	41	Yes	Travel_Rarely	1102	Sales
1	49	No	Travel_Frequently	279	Research & Development
2	37	Yes	Travel_Rarely	1373	Research & Development
3	33	No	Travel_Frequently	1392	Research & Development
4	27	No	Travel_Rarely	591	Research & Development

	DistanceFromHome	Education	EducationField	EmployeeCount
EmployeeNumber \				
0	1	2	Life Sciences	1
1				
1	8	1	Life Sciences	1
2				
2	2	2	Other	1
4				
3	3	4	Life Sciences	1
5				
4	2	1	Medical	1
7				

	RelationshipSatisfaction	StandardHours	StockOptionLevel
...			
0	...	1	80
1	...	4	80
2	...	2	80
3	...	3	80
4	...	4	80

	TotalWorkingYears	TrainingTimesLastYear	WorkLifeBalance
YearsAtCompany \			
0	8	0	1
6			
1	10	3	3
10			
2	7	3	3
0			
3	8	3	3

8			
4	6	3	3
2			

	YearsInCurrentRole	YearsSinceLastPromotion	YearsWithCurrManager
0	4	0	5
1	7	1	7
2	0	0	0
3	7	3	0
4	2	2	2

[5 rows x 35 columns]

Shape:
(1470, 35)

Dataset Information:

<class 'pandas.core.frame.DataFrame'>

RangeIndex: 1470 entries, 0 to 1469

Data columns (total 35 columns):

#	Column	Non-Null Count	Dtype
---	-----	-----	-----
0	Age	1470 non-null	int64
1	Attrition	1470 non-null	object
2	BusinessTravel	1470 non-null	object
3	DailyRate	1470 non-null	int64
4	Department	1470 non-null	object
5	DistanceFromHome	1470 non-null	int64
6	Education	1470 non-null	int64
7	EducationField	1470 non-null	object
8	EmployeeCount	1470 non-null	int64
9	EmployeeNumber	1470 non-null	int64
10	EnvironmentSatisfaction	1470 non-null	int64
11	Gender	1470 non-null	object
12	HourlyRate	1470 non-null	int64
13	JobInvolvement	1470 non-null	int64
14	JobLevel	1470 non-null	int64
15	JobRole	1470 non-null	object
16	JobSatisfaction	1470 non-null	int64
17	MaritalStatus	1470 non-null	object
18	MonthlyIncome	1470 non-null	int64
19	MonthlyRate	1470 non-null	int64
20	NumCompaniesWorked	1470 non-null	int64
21	Over18	1470 non-null	object
22	OverTime	1470 non-null	object
23	PercentSalaryHike	1470 non-null	int64
24	PerformanceRating	1470 non-null	int64
25	RelationshipSatisfaction	1470 non-null	int64
26	StandardHours	1470 non-null	int64
27	StockOptionLevel	1470 non-null	int64

28	TotalWorkingYears	1470	non-null	int64
29	TrainingTimesLastYear	1470	non-null	int64
30	WorkLifeBalance	1470	non-null	int64
31	YearsAtCompany	1470	non-null	int64
32	YearsInCurrentRole	1470	non-null	int64
33	YearsSinceLastPromotion	1470	non-null	int64
34	YearsWithCurrManager	1470	non-null	int64

dtypes: int64(26), object(9)

memory usage: 402.1+ KB

None

Missing Values:

Age	0
Attrition	0
BusinessTravel	0
DailyRate	0
Department	0
DistanceFromHome	0
Education	0
EducationField	0
EmployeeCount	0
EmployeeNumber	0
EnvironmentSatisfaction	0
Gender	0
HourlyRate	0
JobInvolvement	0
JobLevel	0
JobRole	0
JobSatisfaction	0
MaritalStatus	0
MonthlyIncome	0
MonthlyRate	0
NumCompaniesWorked	0
Over18	0
OverTime	0
PercentSalaryHike	0
PerformanceRating	0
RelationshipSatisfaction	0
StandardHours	0
StockOptionLevel	0
TotalWorkingYears	0
TrainingTimesLastYear	0
WorkLifeBalance	0
YearsAtCompany	0
YearsInCurrentRole	0
YearsSinceLastPromotion	0
YearsWithCurrManager	0

dtype: int64

```

# Remove duplicate rows
df = df.drop_duplicates()

# Check duplicates
print("\nDuplicatas after cleaning:")
print(df.duplicated().sum())

Duplicatas after cleaning:
0

# remove unnecessary columns

columns_to_drop = [
    "EmployeeCount",
    "EmployeeNumber",
    "Over18",
    "StandardHours"
]

df = df.drop(columns=columns_to_drop)

print("\nColumns after removing unnecessary columns:")
print(df.columns)

Columns after removing unnecessary columns:
Index(['Age', 'Attrition', 'BusinessTravel', 'DailyRate',
'Department',
'DistanceFromHome', 'Education', 'EducationField',
'EnvironmentSatisfaction', 'Gender', 'HourlyRate',
'JobInvolvement',
'JobLevel', 'JobRole', 'JobSatisfaction', 'MaritalStatus',
'MonthlyIncome', 'MonthlyRate', 'NumCompaniesWorked',
'OverTime',
'PercentSalaryHike', 'PerformanceRating',
'RelationshipSatisfaction',
'StockOptionLevel', 'TotalWorkingYears',
'TrainingTimesLastYear',
'WorkLifeBalance', 'YearsAtCompany', 'YearsInCurrentRole',
'YearsSinceLastPromotion', 'YearsWithCurrManager'],
      dtype='object')

# encode target variable

df["Attrition"] = df["Attrition"].map({
    "Yes": 1,
    "No": 0
})

```

```
print("\nTarget values:")
print(df["Attrition"].value_counts())
```

```
Target values:
Attrition
0      1233
1       237
Name: count, dtype: int64
```

```
# encode categorical variables
```

```
categorical_columns = df.select_dtypes(
    include=["object"]
).columns
```

```
df = pd.get_dummies(
    df,
    columns=categorical_columns,
    drop_first=True
)
```

```
print("\nData after encoding:")
print(df.head())
```

```
Data after encoding:
```

	Age	Attrition	DailyRate	DistanceFromHome	Education	\
0	41	1	1102	1	2	
1	49	0	279	8	1	
2	37	1	1373	2	2	
3	33	0	1392	3	4	
4	27	0	591	2	1	

	EnvironmentSatisfaction	HourlyRate	JobInvolvement	JobLevel	\
0		2	94	3	2
1		3	61	2	2
2		4	92	2	1
3		4	56	3	1
4		1	40	3	1

	JobSatisfaction	...	JobRole_Laboratory Technician
JobRole_Manager	\		
0	4	...	False
False			
1	2	...	False
False			
2	3	...	True
False			
3	3	...	False
False			

```
4          2  ...          True
False
```

```
    JobRole_Manufacturing Director  JobRole_Research Director  \
0                                False                          False
1                                False                          False
2                                False                          False
3                                False                          False
4                                False                          False
```

```
    JobRole_Research Scientist  JobRole_Sales Executive  \
0                                False                      True
1                                True                      False
2                                False                      False
3                                True                      False
4                                False                      False
```

```
    JobRole_Sales Representative  MaritalStatus_Married
MaritalStatus_Single  \
0                                False                      False
True
1                                False                      True
False
2                                False                      False
True
3                                False                      True
False
4                                False                      True
False
```

```
    OverTime_Yes
0            True
1           False
2            True
3            True
4           False
```

```
[5 rows x 45 columns]
```

```
# check data
```

```
print("\nMissing values:")
print(df.isnull().sum())
```

```
print("\nFinal shape:")
print(df.shape)
```

```
Missing values:
Age
```

```
0
```

Attrition	0
DailyRate	0
DistanceFromHome	0
Education	0
EnvironmentSatisfaction	0
HourlyRate	0
JobInvolvement	0
JobLevel	0
JobSatisfaction	0
MonthlyIncome	0
MonthlyRate	0
NumCompaniesWorked	0
PercentSalaryHike	0
PerformanceRating	0
RelationshipSatisfaction	0
StockOptionLevel	0
TotalWorkingYears	0
TrainingTimesLastYear	0
WorkLifeBalance	0
YearsAtCompany	0
YearsInCurrentRole	0
YearsSinceLastPromotion	0
YearsWithCurrManager	0
BusinessTravel_Travel_Frequently	0
BusinessTravel_Travel_Rarely	0
Department_Research & Development	0
Department_Sales	0
EducationField_Life Sciences	0
EducationField_Marketing	0
EducationField_Medical	0
EducationField_Other	0
EducationField_Technical Degree	0
Gender_Male	0
JobRole_Human Resources	0
JobRole_Laboratory Technician	0
JobRole_Manager	0
JobRole_Manufacturing Director	0
JobRole_Research Director	0
JobRole_Research Scientist	0
JobRole_Sales Executive	0
JobRole_Sales Representative	0
MaritalStatus_Married	0
MaritalStatus_Single	0
OverTime_Yes	0
dtype: int64	

Final shape:
(1470, 45)

```
# separate features and target variable
```

```
X = df.drop("Attrition", axis=1)
y = df["Attrition"]
```

Train and compare Logistic Regression, Decision Tree, and Random Forest models.

```
# train test split
```

```
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42,
    stratify=y
)
```

```
print("\nTraining data:", X_train.shape)
print("Testing data:", X_test.shape)
```

```
Training data: (1176, 44)
Testing data: (294, 44)
```

```
# feature scaling
```

```
scaler = StandardScaler()
```

```
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
```

```
# model 1: logistic regression
```

```
logistic_model = LogisticRegression(
    max_iter=1000,
    random_state=42
)
```

```
logistic_model.fit(
    X_train_scaled,
    y_train
)
```

```
logistic_pred = logistic_model.predict(
    X_test_scaled
)
```

```
# model 2: descision tree
```

```
decision_tree = DecisionTreeClassifier(
    criterion="gini",
```

```

        max_depth=5,
        random_state=42
    )

    decision_tree.fit(
        X_train,
        y_train
    )

    tree_pred = decision_tree.predict(
        X_test
    )

# model 3: random forest

    random_forest = RandomForestClassifier(
        n_estimators=100,
        random_state=42
    )

    random_forest.fit(
        X_train,
        y_train
    )

    forest_pred = random_forest.predict(
        X_test
    )

```

Evaluate results using confusion matrix, F1 score, and feature importance visualization.

```

# model evaluation function

def evaluate_model(name, y_test, y_pred):

    print("\n" + name)
    accuracy = accuracy_score(
        y_test,
        y_pred
    )

    precision = precision_score(
        y_test,
        y_pred,
        zero_division=0
    )

    recall = recall_score(
        y_test,
        y_pred,

```

```

        zero_division=0
    )

    f1 = f1_score(
        y_test,
        y_pred,
        zero_division=0
    )

    print("Accuracy :", accuracy)
    print("Precision:", precision)
    print("Recall    :", recall)
    print("F1 Score :", f1)

    print("\nConfusion Matrix:")
    print(confusion_matrix(y_test, y_pred))

    print("\nClassification Report:")
    print(
        classification_report(
            y_test,
            y_pred,
            zero_division=0
        )
    )

    return accuracy, precision, recall, f1

# evaluate all three models

logistic_results = evaluate_model(
    "Logistic Regression",
    y_test,
    logistic_pred
)

tree_results = evaluate_model(
    "Decision Tree",
    y_test,
    tree_pred
)

forest_results = evaluate_model(
    "Random Forest",
    y_test,
    forest_pred
)

```

```

Logistic Regression
Accuracy : 0.8605442176870748

```

Precision: 0.6153846153846154
Recall : 0.3404255319148936
F1 Score : 0.4383561643835616

Confusion Matrix:

```
[[237  10]
 [ 31  16]]
```

Classification Report:

	precision	recall	f1-score	support
0	0.88	0.96	0.92	247
1	0.62	0.34	0.44	47
accuracy			0.86	294
macro avg	0.75	0.65	0.68	294
weighted avg	0.84	0.86	0.84	294

Decision Tree

Accuracy : 0.8333333333333334
Precision: 0.4444444444444444
Recall : 0.1702127659574468
F1 Score : 0.24615384615384617

Confusion Matrix:

```
[[237  10]
 [ 39   8]]
```

Classification Report:

	precision	recall	f1-score	support
0	0.86	0.96	0.91	247
1	0.44	0.17	0.25	47
accuracy			0.83	294
macro avg	0.65	0.56	0.58	294
weighted avg	0.79	0.83	0.80	294

Random Forest

Accuracy : 0.8333333333333334
Precision: 0.4166666666666667
Recall : 0.10638297872340426
F1 Score : 0.1694915254237288

Confusion Matrix:

```
[[240   7]
 [ 42   5]]
```

Classification Report:

	precision	recall	f1-score	support
0	0.85	0.97	0.91	247
1	0.42	0.11	0.17	47
accuracy			0.83	294
macro avg	0.63	0.54	0.54	294
weighted avg	0.78	0.83	0.79	294

```
# model comparison
```

```
results = pd.DataFrame({
```

```
    "Model": [
        "Logistic Regression",
        "Decision Tree",
        "Random Forest"
    ],
```

```
    "Accuracy": [
        logistic_results[0],
        tree_results[0],
        forest_results[0]
    ],
```

```
    "Precision": [
        logistic_results[1],
        tree_results[1],
        forest_results[1]
    ],
```

```
    "Recall": [
        logistic_results[2],
        tree_results[2],
        forest_results[2]
    ],
```

```
    "F1 Score": [
        logistic_results[3],
        tree_results[3],
        forest_results[3]
    ]
}
```

```
})
```

```
print("MODEL COMPARISON")
```

```
print(results)
```

```
MODEL COMPARISON
```

```
Model Accuracy Precision Recall F1 Score
```

0	Logistic Regression	0.860544	0.615385	0.340426	0.438356
1	Decision Tree	0.833333	0.444444	0.170213	0.246154
2	Random Forest	0.833333	0.416667	0.106383	0.169492

```
# F1 score comparison
```

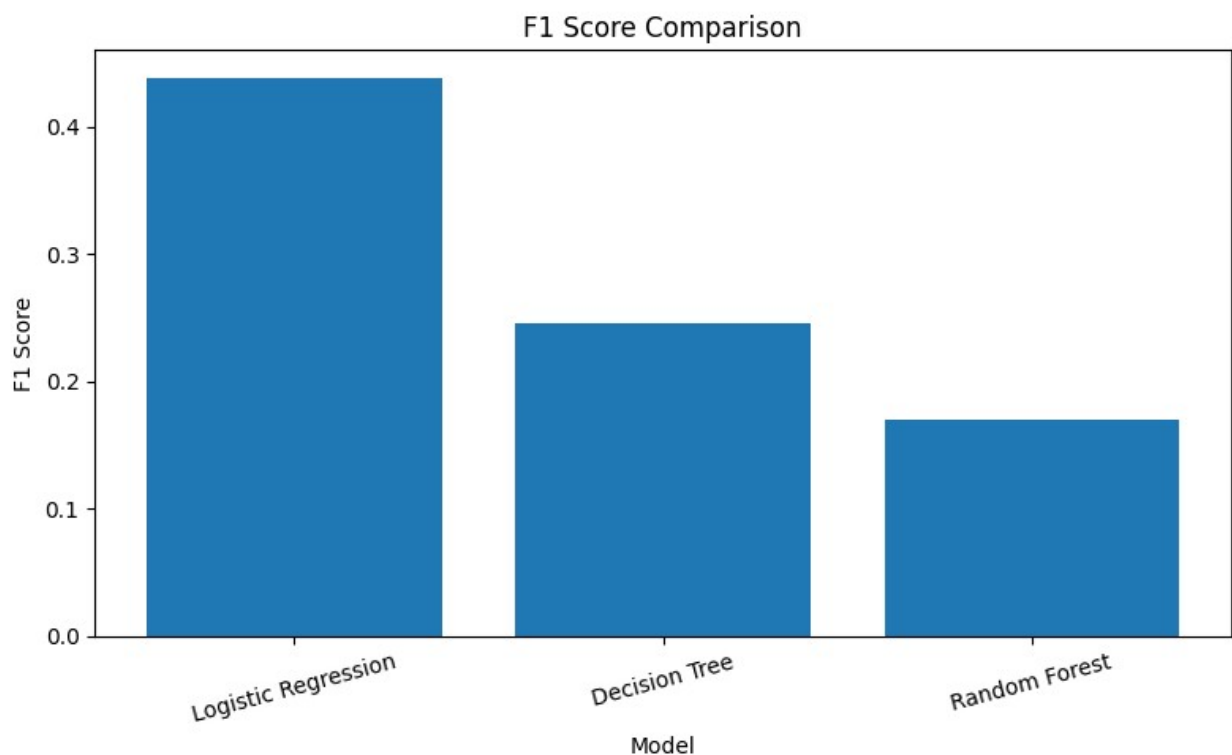
```
plt.figure(figsize=(8, 5))

plt.bar(
    results["Model"],
    results["F1 Score"]
)

plt.xlabel("Model")
plt.ylabel("F1 Score")
plt.title("F1 Score Comparison")

plt.xticks(rotation=15)

plt.tight_layout()
plt.show()
```



```
# confusion matrix for logistic regression
```

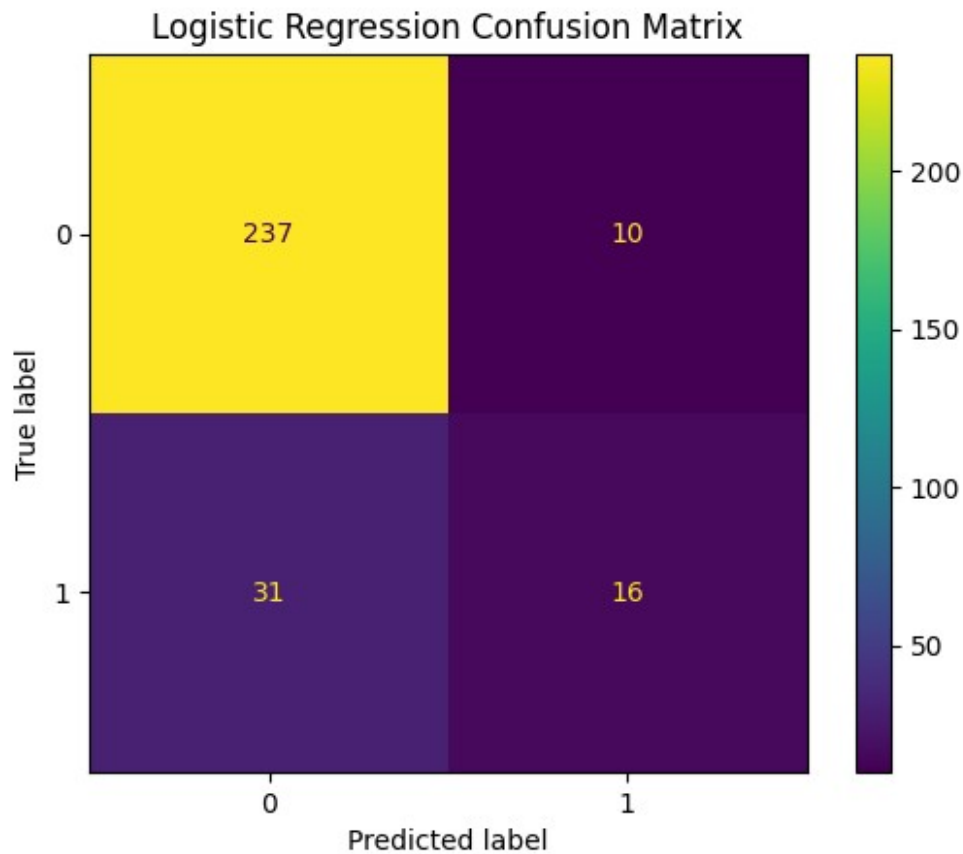
```
ConfusionMatrixDisplay.from_predictions(
    y_test,
```

```

    logistic_pred
)

plt.title("Logistic Regression Confusion Matrix")
plt.show()

```



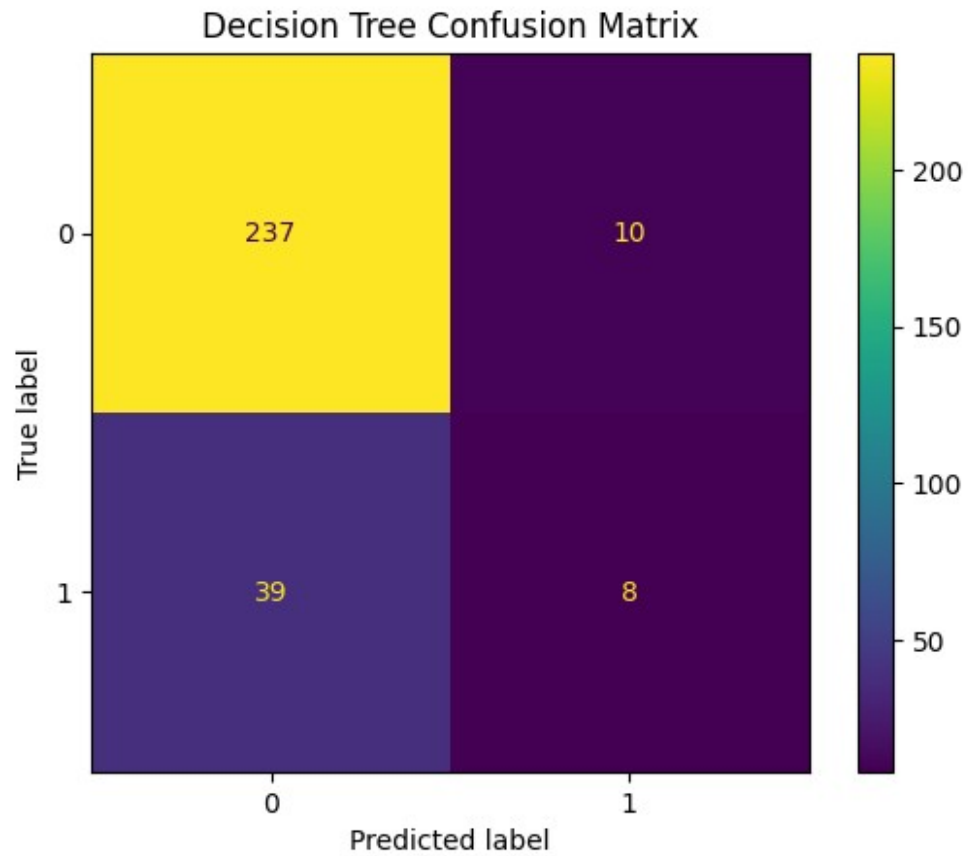
```

# confusion matrix for decision tree

ConfusionMatrixDisplay.from_predictions(
    y_test,
    tree_pred
)

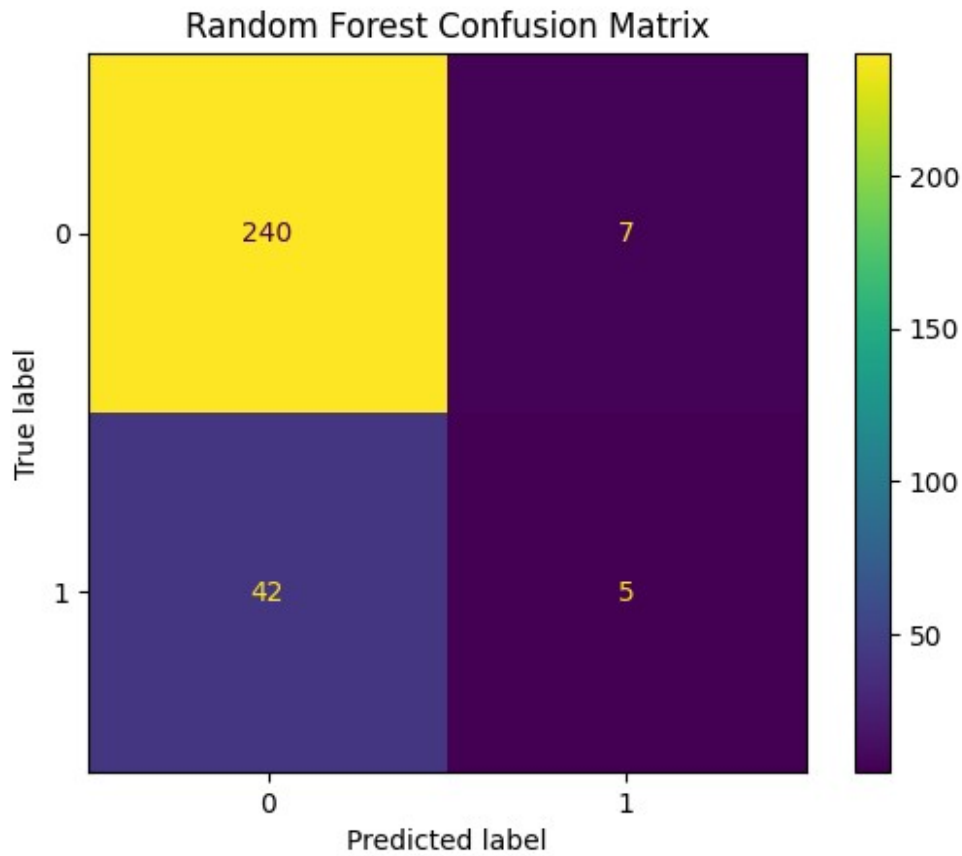
plt.title("Decision Tree Confusion Matrix")
plt.show()

```



```
# confusion matrix for random forest
ConfusionMatrixDisplay.from_predictions(
    y_test,
    forest_pred
)

plt.title("Random Forest Confusion Matrix")
plt.show()
```



```
# random forest feature importance
importance = random_forest.feature_importances_

feature_importance = pd.DataFrame({
    "Feature": X.columns,
    "Importance": importance
})

# Sort from highest to lowest
feature_importance = feature_importance.sort_values(
    by="Importance",
    ascending=False
)

print("TOP IMPORTANT FEATURES")

print(
    feature_importance.head(15)
)

TOP IMPORTANT FEATURES
      Feature  Importance
```

9	MonthlyIncome	0.072306
0	Age	0.072260
16	TotalWorkingYears	0.053589
1	DailyRate	0.051670
5	HourlyRate	0.047866
2	DistanceFromHome	0.047793
10	MonthlyRate	0.047228
19	YearsAtCompany	0.044137
43	OverTime_Yes	0.037830
22	YearsWithCurrManager	0.035452
11	NumCompaniesWorked	0.035212
12	PercentSalaryHike	0.033997
4	EnvironmentSatisfaction	0.027282
17	TrainingTimesLastYear	0.026506
20	YearsInCurrentRole	0.026050

feature importance visualization

```
top_features = feature_importance.head(15)
```

```
plt.figure(figsize=(10, 6))
```

```
plt.barh(
    top_features["Feature"],
    top_features["Importance"]
)
```

```
plt.xlabel("Importance")
```

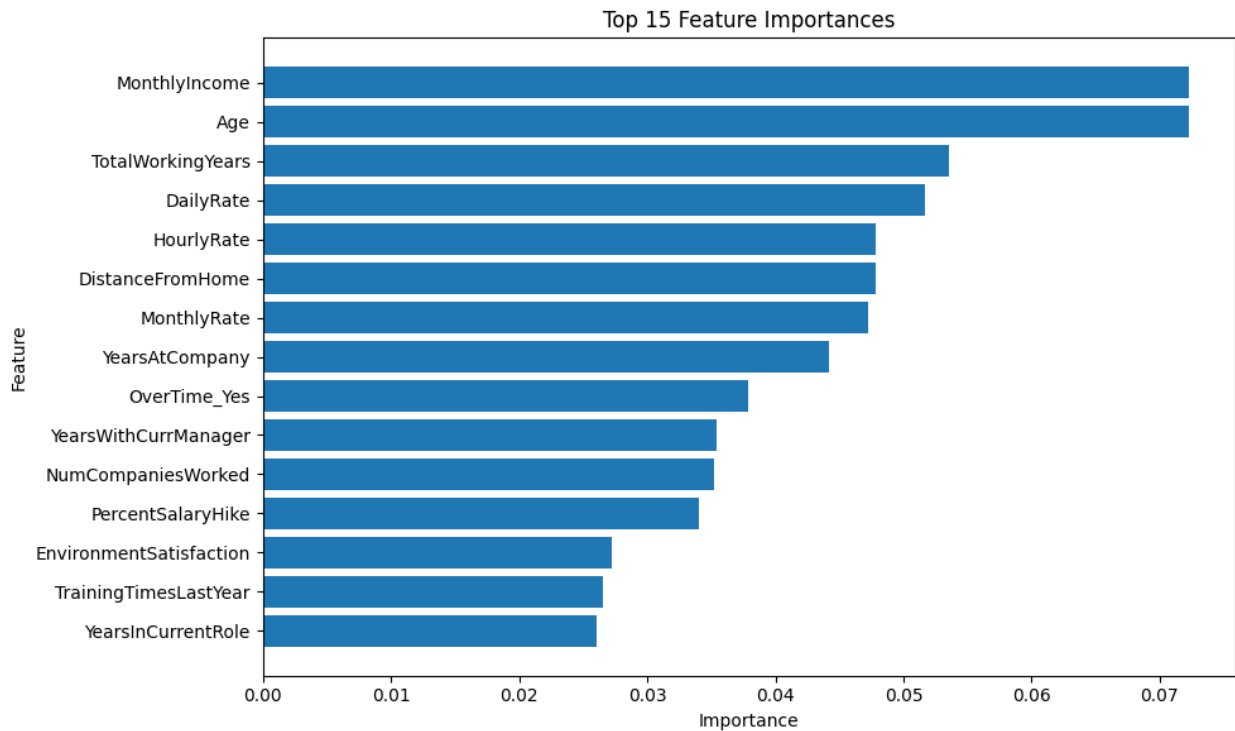
```
plt.ylabel("Feature")
```

```
plt.title("Top 15 Feature Importances")
```

```
plt.gca().invert_yaxis()
```

```
plt.tight_layout()
```

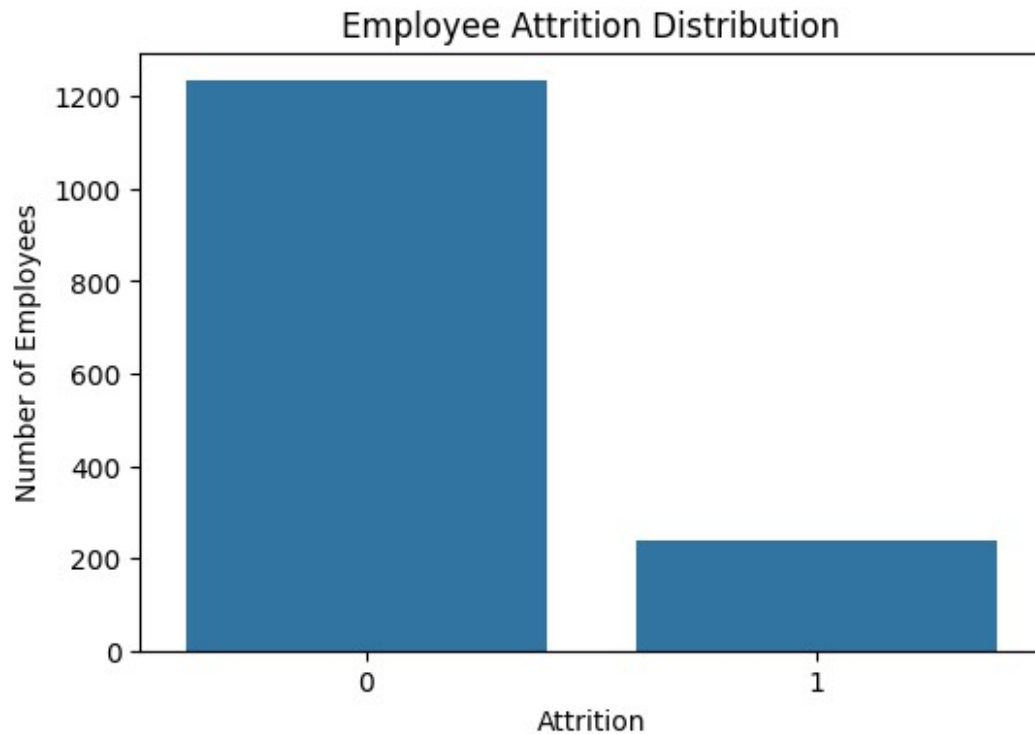
```
plt.show()
```



```
# attrition distribution
plt.figure(figsize=(6, 4))
sns.countplot(
    x=df["Attrition"]
)

plt.xlabel("Attrition")
plt.ylabel("Number of Employees")
plt.title("Employee Attrition Distribution")

plt.show()
```



```
# final result

best_model = results.loc[
    results["F1 Score"].idxmax(),
    "Model"
]

best_f1 = results["F1 Score"].max()

print("FINAL RESULT")

print("Best Model:", best_model)
print("Best F1 Score:", best_f1)

FINAL RESULT
Best Model: Logistic Regression
Best F1 Score: 0.4383561643835616
```